

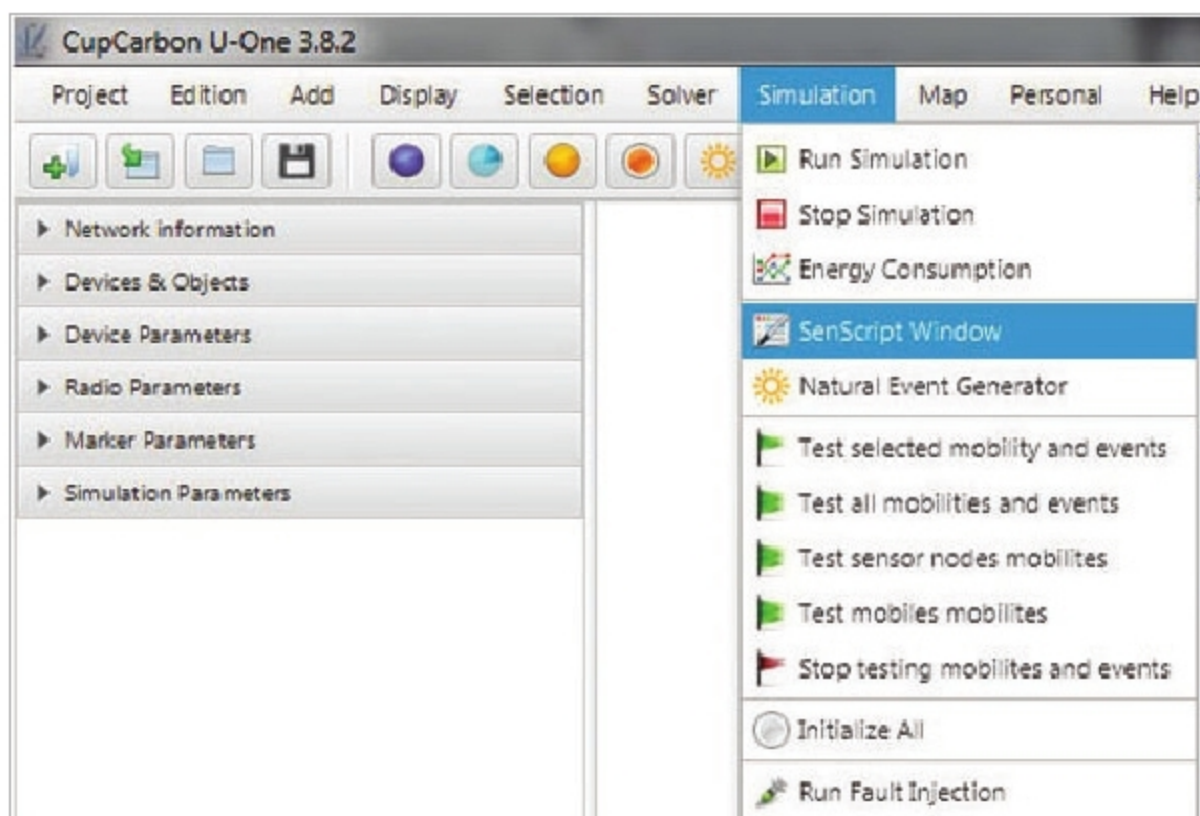


Figure 6: Option for the SenScript window in CupCarbon

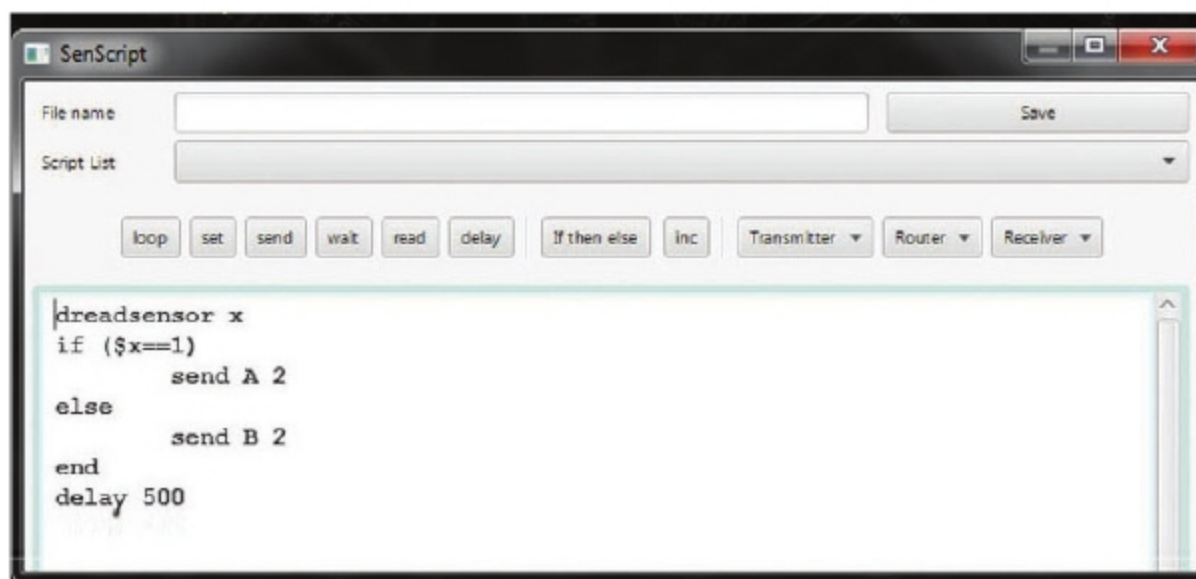


Figure 7: The SenScript Editor in CupCarbon for programming of sensors

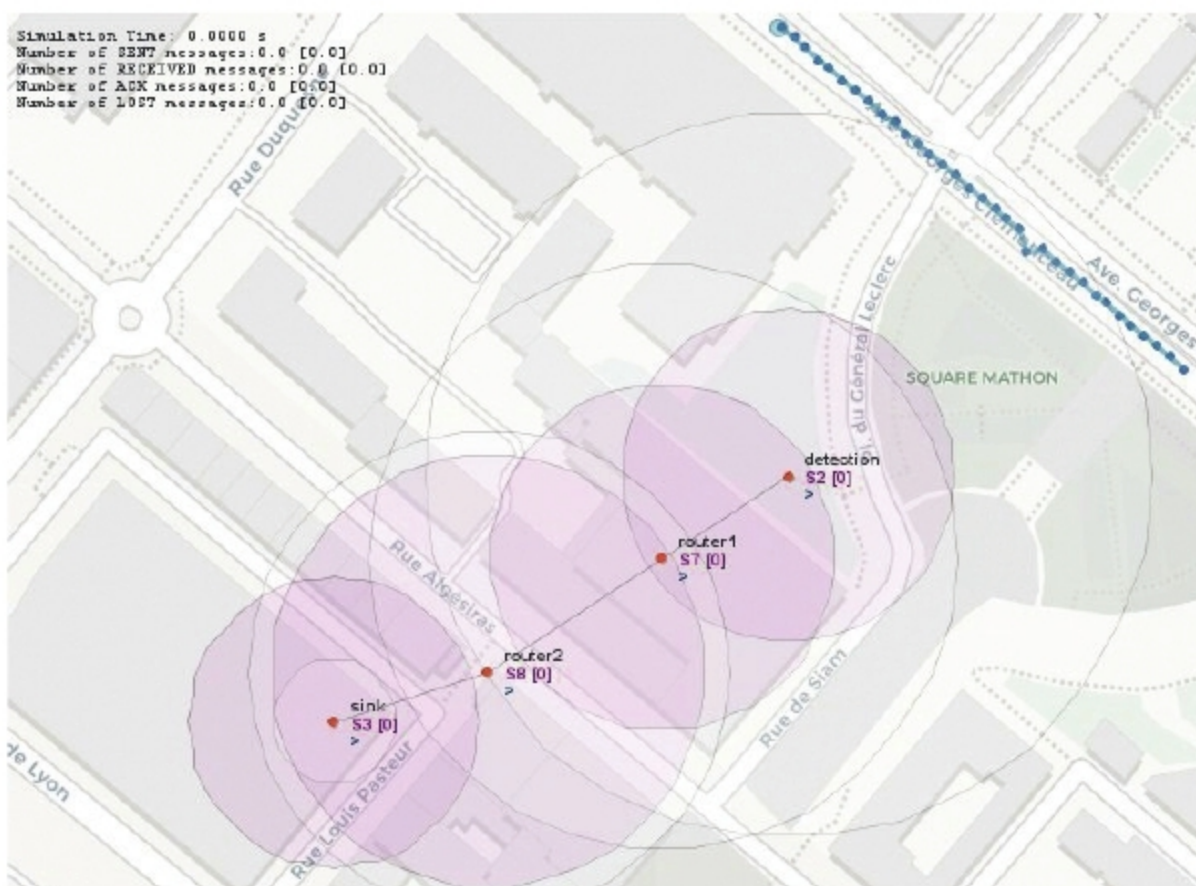


Figure 8: Integration of markers and routes in CupCarbon

simulation environment. In SenScript, a number of programming constructs and modules can be used so that the smart city environment can be simulated.

Creating dynamic scenarios for IoT and smart cities using the CupCarbon simulator

The working environment of CupCarbon has numerous options to create and program sensors of different types. In the middle, there is a Map View, in which the smart city under simulation can be viewed dynamically.

The sensors and smart objects are displayed in Map View. To program these smart devices and traffic objects, the toolbar of CupCarbon provides the programming modules so that the behaviour of every object can be controlled.

Any number of nodes or motes can be imported in CupCarbon to program them in random positions. In addition, the weather conditions and environment factors can be added so that the smart city project can be simulated under specific environmental conditions. Using this option, the performance of the smart city can be evaluated under different situations with varying city temperatures.

The SenScript editor provides the programming editor so that the functions and methods with each sensor or smart device can be executed. This editor has a wide range of inbuilt functions which can be called. These functions can be attached to the sensors and smart objects in the CupCarbon simulator.

The markers and routes provide the traffic path for the vehicles in the smart city, so that these can follow the shortest path from source to destination, factoring in congestion or traffic jams.

On executing the code written in SenScript, an animated view of the smart city is produced, representing the mobility of vehicles, persons and traffic objects. This view enables the development team to check whether there is any probability of congestion or loss of performance. Using this process of visualisation, the algorithms and associated code of SenScript can